

Natural Language Processing

11. Large Language Model

CBNU

School of Computer Science

Emeritus Prof. Lee, Jae Sung



Overview

❖ What we will learn today

- Pre-training & Tokenization
- Tokenization Methods (BPE, WordPiece)
- Major LLM Architectures (BERT, GPT, T5)

1. Pre-Training and Tokenization

What is an LLM?

❖ Large Language Model

- A neural network trained on massive text data
- Typically based on Transformer architecture
- Parameters: millions → billions → trillions

❖ Capabilities:

- Text generation (ChatGPT)
- Question answering
- Translation
- Summarization

❖ Key idea:

- “Predict the next token based on context”

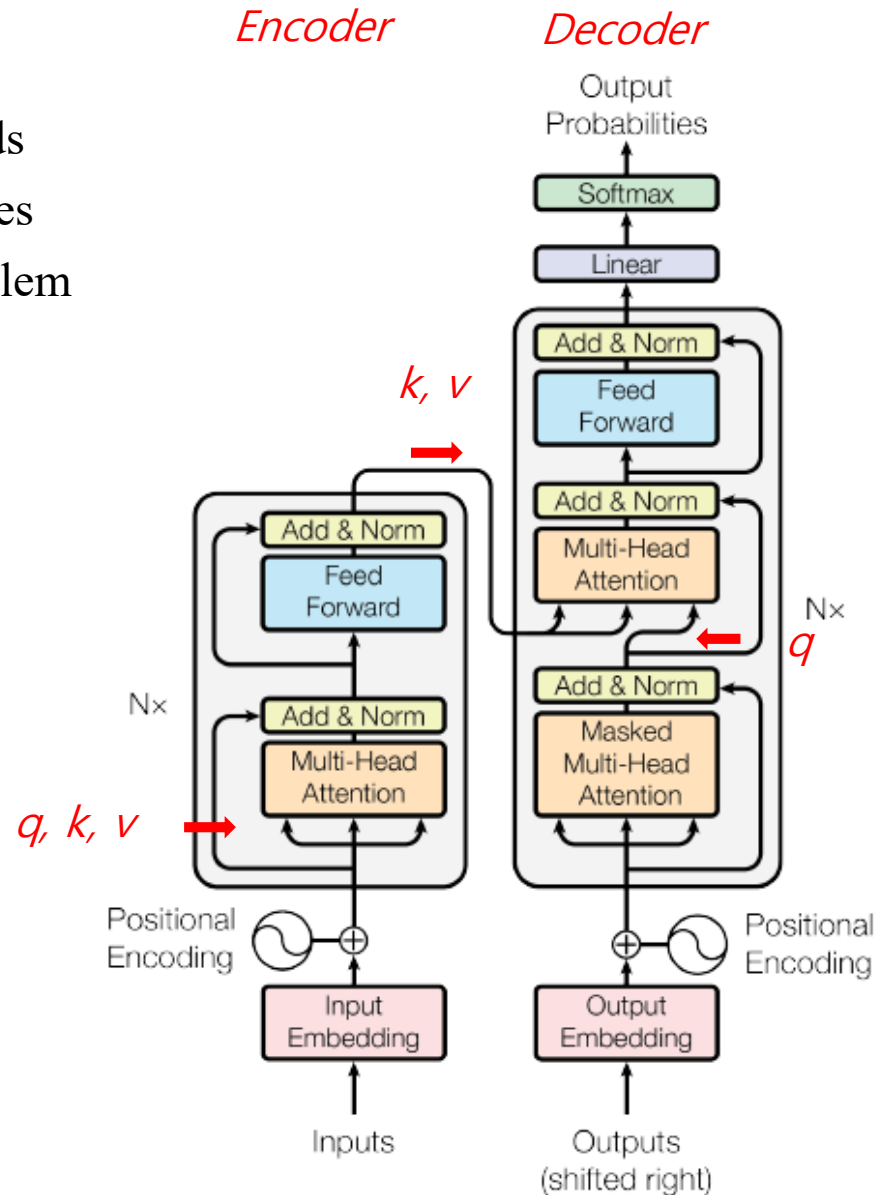
Transformer Review (Core Idea)

❖ Self-Attention Mechanism

- Each word attends to all other words
- Captures long-distance dependencies
- Solves information bottleneck problem

❖ Advantages

- Parallel computation
- Deep stacking (multi-layer)
- Flexible architecture:
 - Encoder (BERT)
 - Decoder (GPT)
 - Encoder-Decoder (T5)



Why Pre-training?

❖ Problem: Lack of labeled data

- Supervised data is expensive
- Language understanding requires huge data

❖ Solution: Pre-training

- Train on raw text (self-supervised learning)
- Learn general language knowledge

❖ Then apply:

- Fine-tuning for specific tasks

Self-Supervised Learning Tasks

❖ Examples:

- Next word prediction
→ “I go to school to ____”
- Masked word prediction
→ “I drank ____ at a cafe”
- Sentence order prediction
- Next sentence prediction

❖ Insight:

- The model creates its own labels from raw text

What is Tokenization?

❖ **Tokenization = splitting text into units**

- Word → subword → characters

❖ **Why needed:**

- Vocabulary size limitation
- Handle unknown words (OOV problem)
- Reduce input dimensionality

Subword Tokenization

❖ **Key idea: Break words into smaller units**

❖ **Examples:**

- lovely → love + ly
- wonderful → wonder + ful
- undo → un + do
- => unthankfully → un + thank + ful + ly

❖ **Benefits:**

- Better generalization
- Efficient vocabulary (~30K–100K tokens)

Byte Pair Encoding (BPE)

❖ Frequency-based merging

❖ Process:

1. Start with characters
2. Count frequent pairs (bigrams)
3. Merge most frequent pairs
4. Repeat

❖ Benefits:

- Simple and effective
- Used in GPT models

BPE Example (Intuition)

❖ Intuition

- Frequent patterns become tokens
- Rare words are decomposed

❖ Result:

- Fewer tokens overall
- Better compression

BPE Voc Construction Example Step 1 & 2

❖ BPE vocabulary construction example (voc size = 10)

Step 1

1. List words delimited by spaces

```
bug hug bun fun  bug ...  
bug bun ...fun  ....
```

2. Count frequency

```
hug 10  
bug 5  
bun 7  
fun 6
```

3. Store unigrams in Dic

```
h, u, g, b,  
n, f
```

Step 2

1. List unigrams

```
h, u, g 10  
b, u, g 5  
b, u, n 7  
f, u, n 6
```

2. Combine bigrams

```
h, u 10  
u, g 10  
b, u 5  
u, g 5  
b, u 7  
u, n 7  
f, u 6  
u, n 6
```

3. Choose the most frequent bigram

```
h, u 10  
u, g 15  
b, u 12  
u, n 13  
f, u 6
```

4. Store the bigram

```
h, u, g, b,  
n, f,  
ug
```

BPE Voc Construction Example Step 3 & 4

Step 3

1. List unigrams

h, ug	10
b, ug	5
b, u, n	7
f, u, n	6

2. Combine bigrams

h, ug	10
b, ug	5
b, u	7
u, n	7
f, u	6
u, n	6

3. Choose the most frequent bigram

h, ug	10
b, ug	5
b, u	7
u, n	13
f, u	6

4. Store the bigram

h, u, g, b,
n, f,
ug, un

Step 4

1. List unigrams

h, ug	10
b, ug	5
b, un	7
f, un	6

2. Combine bigrams

h, ug	10
b, ug	5
b, un	7
f, un	6

3. Choose the most frequent bigram

h, ug	10
b, ug	5
b, un	7
f, un	6

4. Store the bigram

h, u, g, b,
n, f,
ug, un,
hug

BPE Voc Construction Example Step 5

Step 5

1. List unigrams

hug	10
b, ug	5
b, un	7
f, un	6

2. Combine bigrams

b, ug	5
b, un	7
f, un	6

3. Choose the most frequent bigram

b, ug	5
b, un	7
f, un	6

4. Store the bigram

h, u, g, b,
n, f,
ug, un,
hug, bun

$|V| = 10$

BPE Tokenizing Example

❖ Tokenization

- Tokenize using token vocabulary (constructed before)
- Segment input text into unigram characters
- Combine bigrams which are registered in Voc
- Unregistered grams are marked as <unk>
- Repeat bigram combining until no more new registered bigrams are found.

Voc: ug, un, hug, bun, h, u, g, b, n, f

bug -> b, u, g -> b, ug

mug -> m, u, g -> <unk>, ug

hug -> h, u, g -> h, ug -> hug

WordPiece Tokenization

- ❖ **Improved version of BPE**
- ❖ **Differences:**
 - Uses probability (mutual information)
 - Chooses longest matching token
 - Distinguishes word boundary
- ❖ **Example:**
 - playing → play + ##ing
 - unfortunately → un + ##fortunate + ##ly
- ❖ **Used in BERT**

2. Large Language Model (LLM)

Types of LLMs

❖ Auto-Encoding Models

- Focus: representation learning
- Task: sequence labeling
- Examples:
 - ELMo
 - BERT

❖ Auto-Regressive Models

- Focus: text generation
- Task: sequence generation
- Examples:
 - GPT series

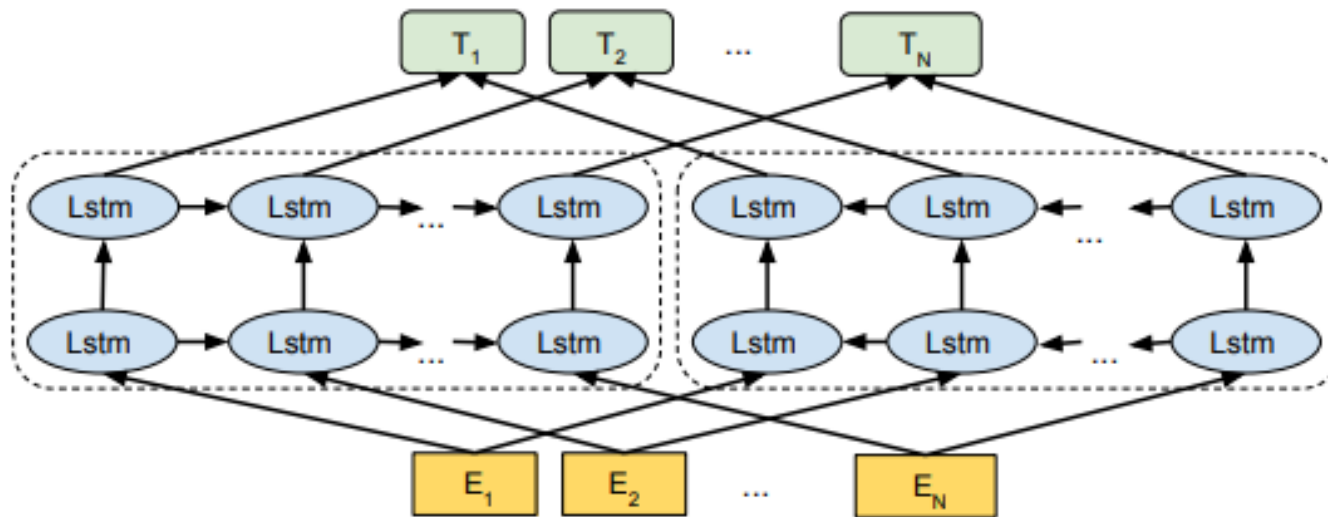
ELMO

❖ ELMo (Embeddings from Language Models)

- Bi-LSTM based
- Learns forward + backward context
- Produces context-dependent embeddings

❖ Key idea:

→ Same word → different embeddings depending on context

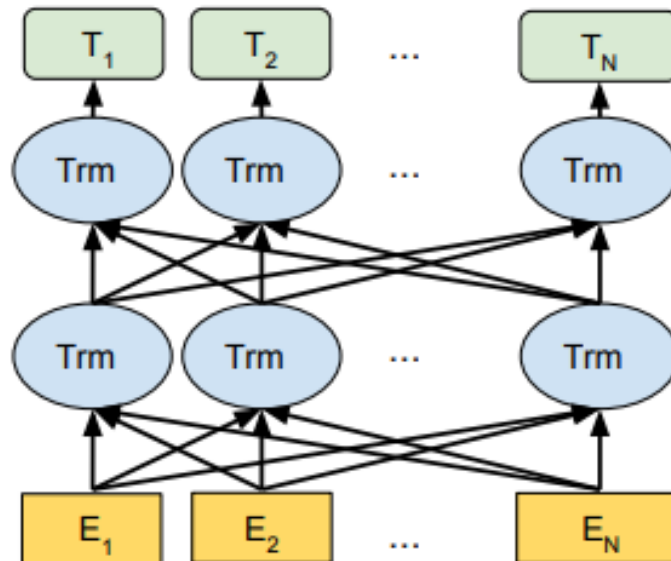


ELMo Architecture

BERT 모델

❖ BERT: Bi-directional Encoder Representations from Transformers

- Bidirectional Transformer encoder
- Used for sequence labeling tasks



BERT Tokenization

❖ Tokenizer

- Uses WordPiece
- Subword-based
- Reduces unknown words

❖ Example

- playing -> play ##ing
- unfortunately -> un ##fortunate ##ly

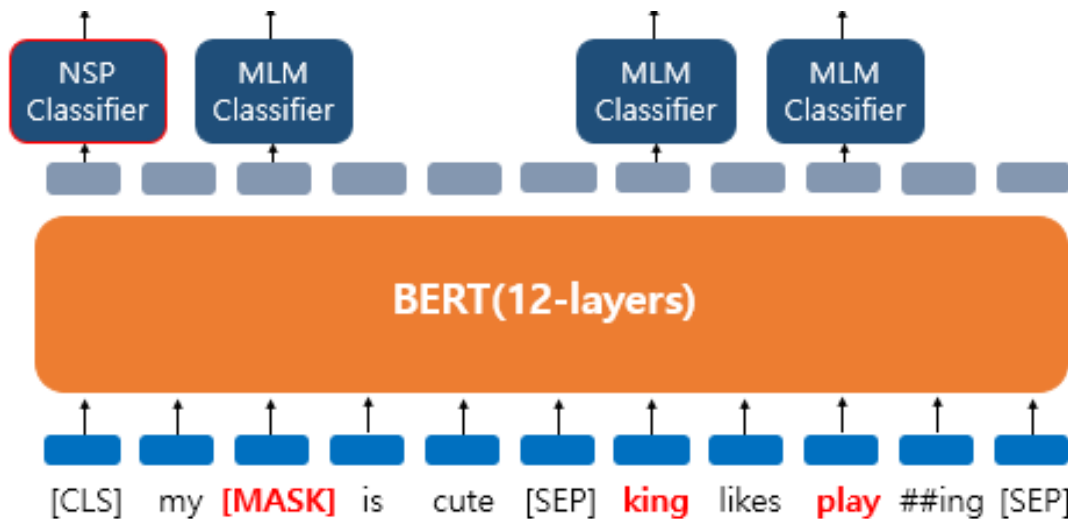
BERT Pre-training

❖ Masked Language Modeling

- Predict masked tokens
- 15% tokens selected
- 80% [mask] , 10% random, 10% no change

❖ Next Sentence Prediction

- Learn relationship between sentences



source: my dog is cute. he likes playing

BERT fine tuning

❖ Fine tuning task

- Adapt model to specific tasks
- Use labeled data

❖ Fine tuning task list 1

- MNLI (Multi-Genre Natural Language Inference)
 - Entailment classification task
- QQP (Quora Question Pairs)
 - Check the semantically same of Quora question pairs
- QNLI (Question Natural Language Inference)
 - binary classification of SQuAD. Answer inclusion in paragraph
- STS-B (Semantic Textual Similarity Benchmark)
 - Check sentence similarity

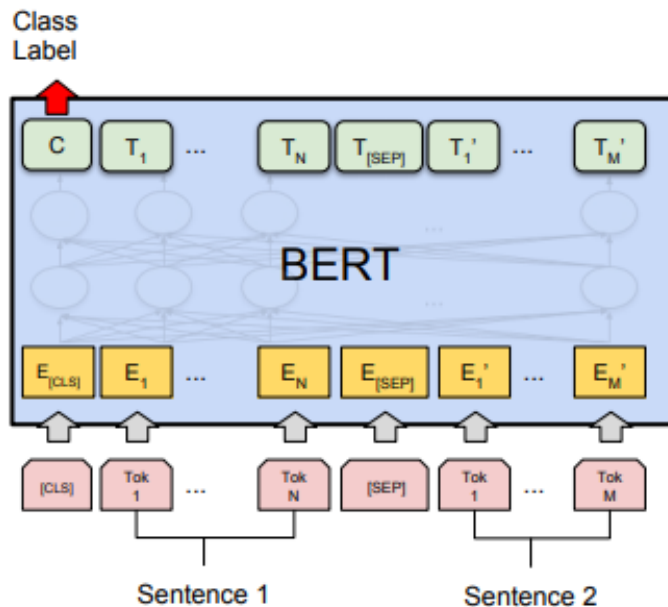
BERT fine tuning

❖ Fine tuning task list 2

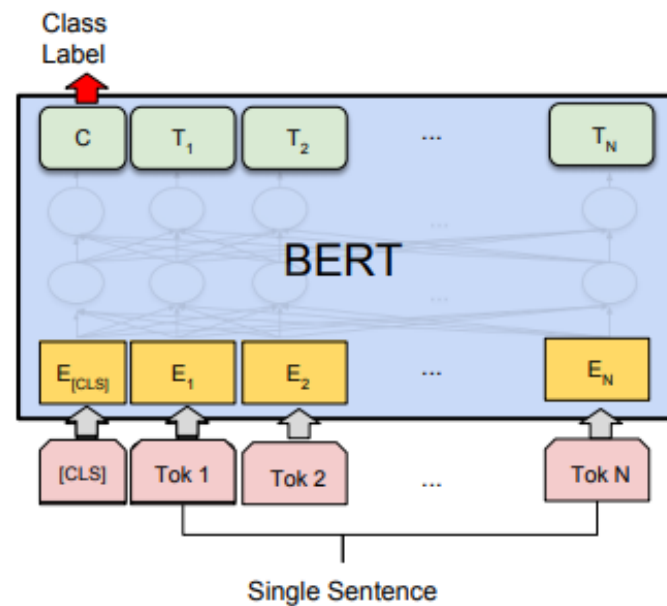
- MRPC (Microsoft Research Paraphrase Corpus)
 - Sentence pair similarity
- RTE (Recognizing Textual Entailment)
 - Similar to MNLI (entailment classification)
- SST-2 (Stanford Sentiment Treebank)
 - Single sentence binary classification.
 - Movie review: sentiment is marked
- CoLA (Corpus of Linguistic Acceptability)
 - Binary classification for English sentence correctness in grammar
- SWAG (Situations With Adversarial Generations)
 - Choose one most natural following sentence among 4 for given context
- SQuAD (Stanford Question Answering Dataset)
 - Machine reading comprehension: find answer text in source for the given question

BERT multitask model

❖ BERT multitask model 1



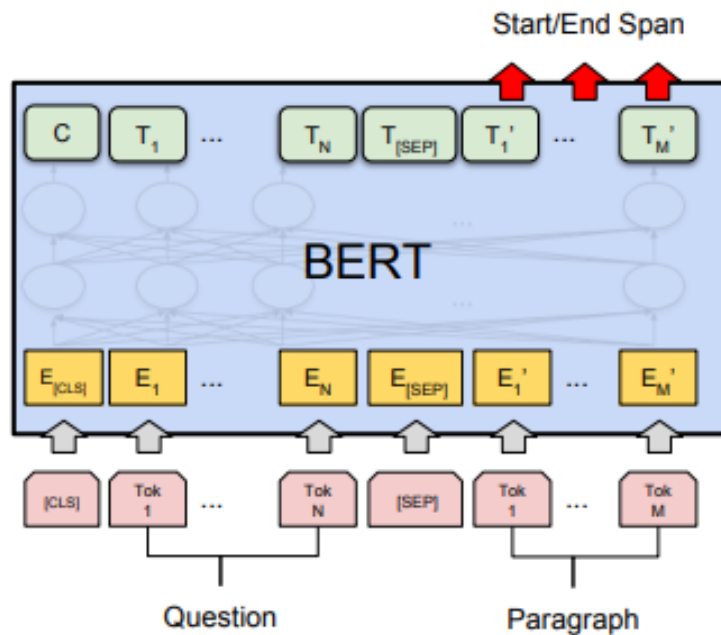
(a) Sentence Pair Classification Tasks:
MNLI, QQP, QNLI, STS-B, MRPC,
RTE, SWAG



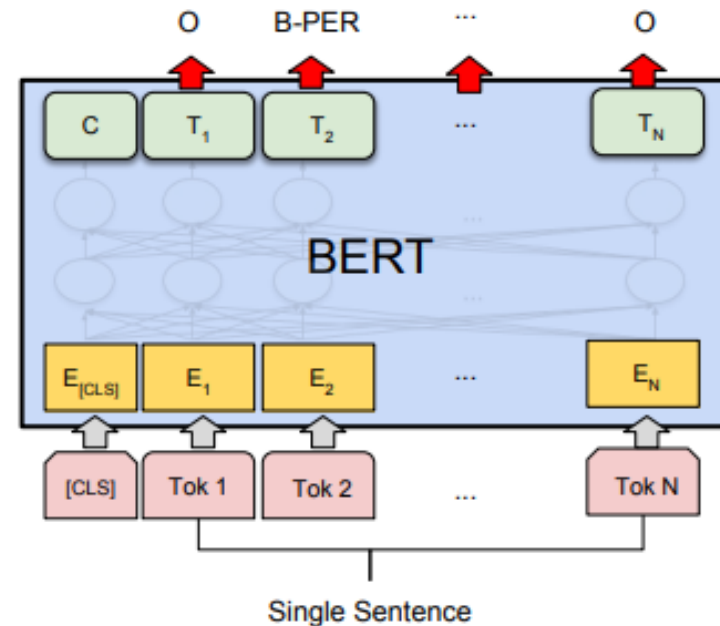
(b) Single Sentence Classification Tasks:
SST-2, CoLA

BERT Multitask Model

❖ BERT multitask model 2



(c) Question Answering Tasks:
SQuAD v1.1



(d) Single Sentence Tagging Tasks:
CoNLL-2003 NER

BERT Performance

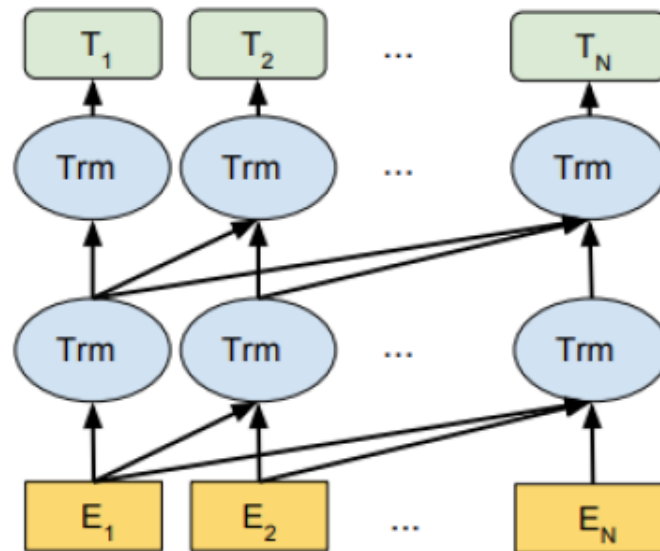
❖ Performance comparison (Devlin et al., 2018)

System	MNLI-(m/mm) 392k	QQP 363k	QNLI 108k	SST-2 67k	CoLA 8.5k	STS-B 5.7k	MRPC 3.5k	RTE 2.5k	Average -
Pre-OpenAI SOTA	80.6/80.1	66.1	82.3	93.2	35.0	81.0	86.0	61.7	74.0
BiLSTM+ELMo+Attn	76.4/76.1	64.8	79.8	90.4	36.0	73.3	84.9	56.8	71.0
OpenAI GPT	82.1/81.4	70.3	87.4	91.3	45.4	80.0	82.3	56.0	75.1
BERT _{BASE}	84.6/83.4	71.2	90.5	93.5	52.1	85.8	88.9	66.4	79.6
BERT _{LARGE}	86.7/85.9	72.1	92.7	94.9	60.5	86.5	89.3	70.1	82.1

GPT Models

❖ GPT-x: Generative Pre-Training 1, 2, 3

- Unidirectional Transformer decoder
- Auto regressive
- sequence-to-sequence



GPT tokenizer and pre-training

❖ Tokenizer

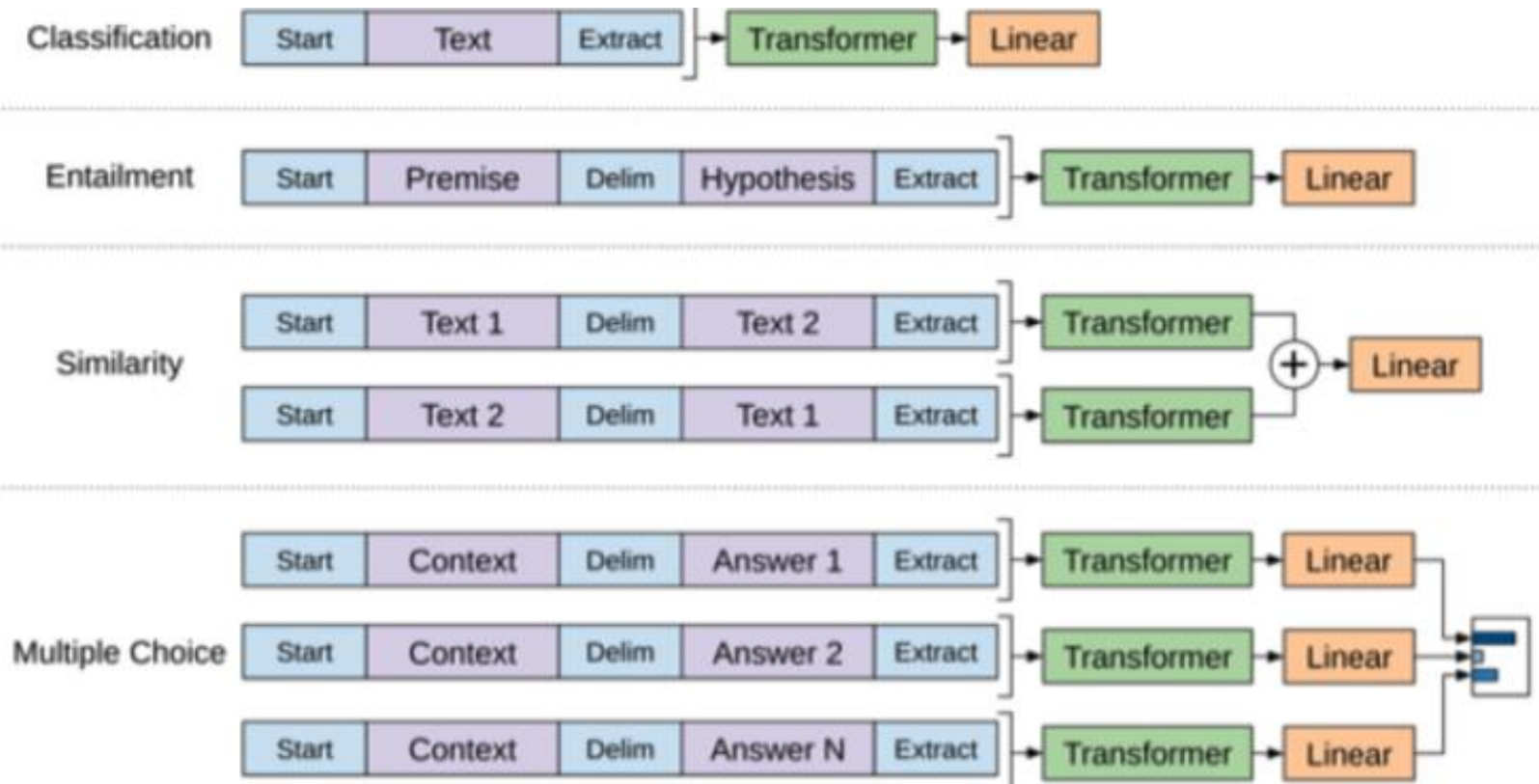
- Byte Pair Encoding
- Subword based on frequency
- Mitigate unknown word problem

❖ Pretraining

- Next word prediction

GPT fine tuning

❖ GPT-1 fine tuning



GPT Performance

❖ GPT-1 Performance (Radford et al., 2018)

Method	MNLI-m	MNLI-mm	SNLI	SciTail	QNLI	RTE
ESIM + ELMo [44] (5x)	-	-	<u>89.3</u>	-	-	-
CAFE [58] (5x)	80.2	79.0	<u>89.3</u>	-	-	-
Stochastic Answer Network [35] (3x)	<u>80.6</u>	<u>80.1</u>	-	-	-	-
CAFE [58]	78.7	77.9	88.5	<u>83.3</u>		
GenSen [64]	71.4	71.3	-	-	<u>82.3</u>	59.2
Multi-task BiLSTM + Attn [64]	72.2	72.1	-	-	82.1	61.7
Finetuned Transformer LM (ours)	82.1	81.4	89.9	88.3	88.1	56.0

GPT parameters and data

❖ GPT-3 parameters

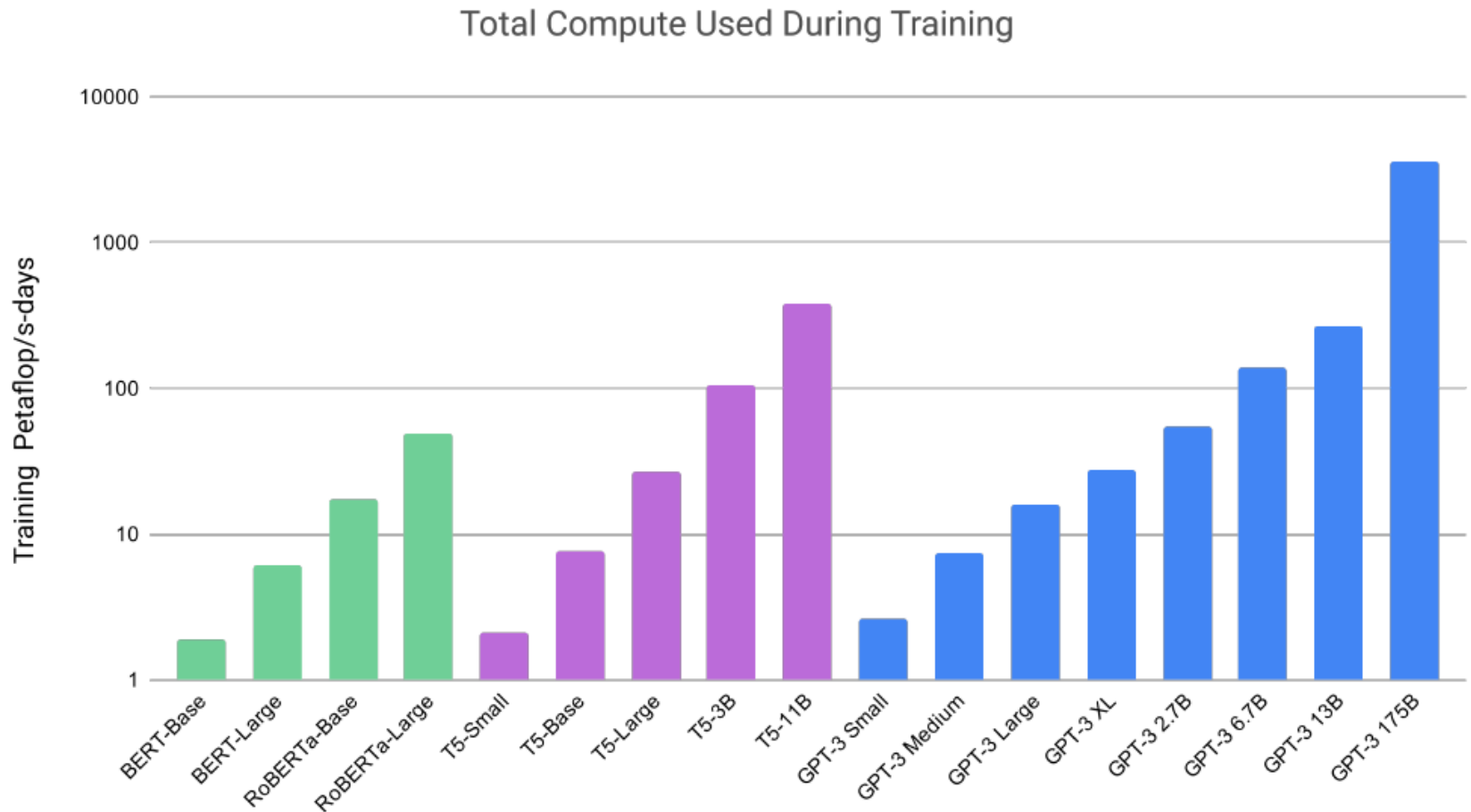
Model Name	n_{params}	n_{layers}	d_{model}	n_{heads}	d_{head}	Batch Size	Learning Rate
GPT-3 Small	125M	12	768	12	64	0.5M	6.0×10^{-4}
GPT-3 Medium	350M	24	1024	16	64	0.5M	3.0×10^{-4}
GPT-3 Large	760M	24	1536	16	96	0.5M	2.5×10^{-4}
GPT-3 XL	1.3B	24	2048	24	128	1M	2.0×10^{-4}
GPT-3 2.7B	2.7B	32	2560	32	80	1M	1.6×10^{-4}
GPT-3 6.7B	6.7B	32	4096	32	128	2M	1.2×10^{-4}
GPT-3 13B	13.0B	40	5140	40	128	2M	1.0×10^{-4}
GPT-3 175B or “GPT-3”	175.0B	96	12288	96	128	3.2M	0.6×10^{-4}

❖ GPT-3 training data(total 499B)

Dataset	Quantity (tokens)	Weight in training mix	Epochs elapsed when training for 300B tokens
Common Crawl (filtered)	410 billion	60%	0.44
WebText2	19 billion	22%	2.9
Books1	12 billion	8%	1.9
Books2	55 billion	8%	0.43
Wikipedia	3 billion	3%	3.4

GPT Performance

❖ GPT-3 Computing time



* PetaFLOP/s-days: a unit for amount of communication. A unit of throughput is 10^{15} FLOP/s * 3600 s/hr * 24 hr/day. FLOP: FLOating point OPerations. Nvidia H100 SXM: 67 Tflops. RTX A6000 38 Tflops. RTX A100: 19 Tflops.

T5 (Text-to-Text Transfer Transformer)

- ❖ **Optimizing Existing LLM Models**
- ❖ Explore optimal models through various combinations of:
 - model architectures
 - training datasets
- ❖ Training strategies:
 - Unsupervised pre-training
 - Multi-task pre-training
 - Fine-tuning
- ❖ Key factors:
 - Model size
 - Number of training steps
 - Batch size
 - Multi-model approaches (e.g., ensemble)

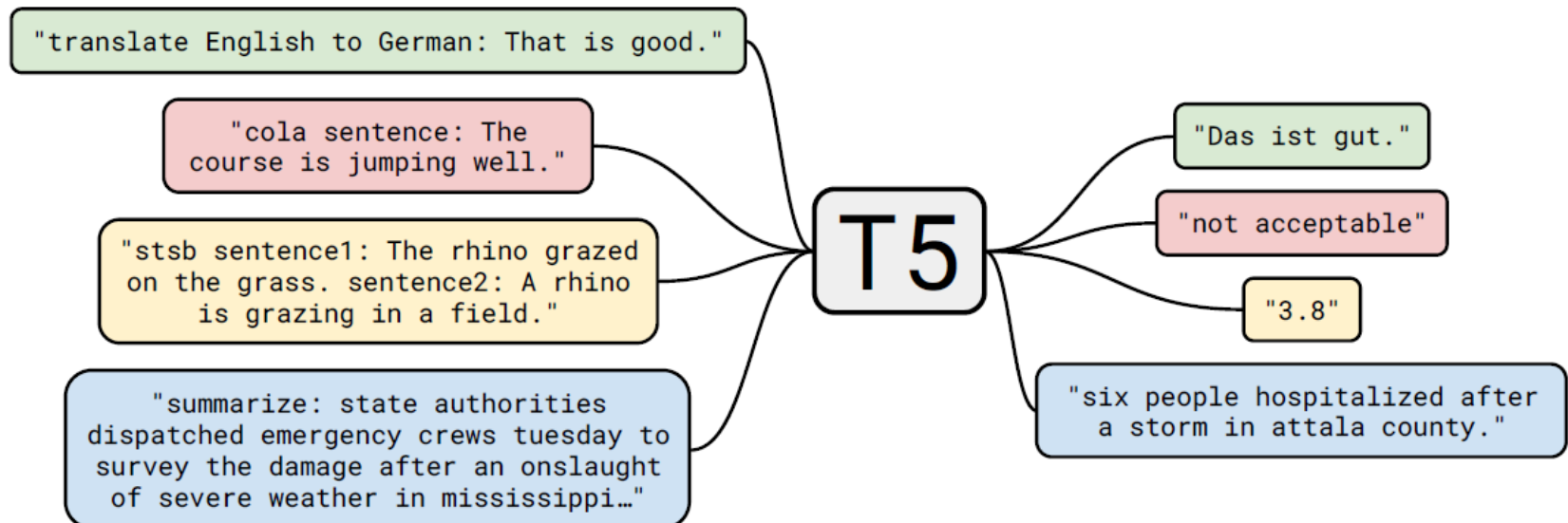
T5 (Text-to-Text Transfer Transformer)

❖ Text-to-Text Framework

- Convert all tasks into text format
- Use a unified input/output representation for consistency

❖ Representing Numbers as Text

- Numerical values are also expressed as text
- Example:
 - Similarity scores represented as text labels (e.g., 1 to 5)



T5 Pre-training

❖ Pre-training

- Multiple token mask

Original text

Thank you ~~for~~ ~~inviting~~ me to your party ~~last~~ week.

Inputs

Thank you <X> me to your party <Y> week.

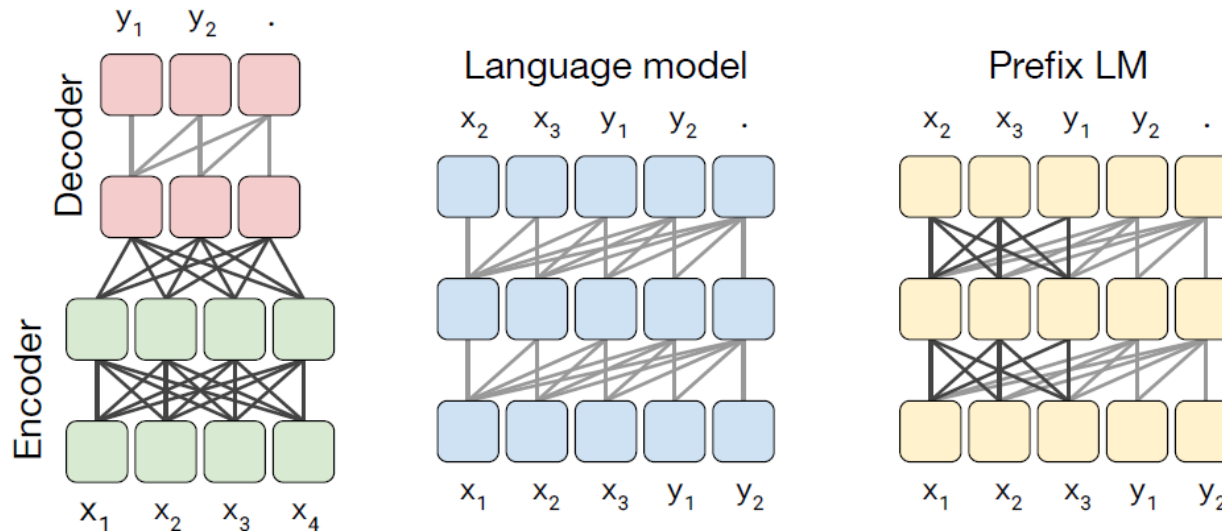
Targets

<X> for inviting <Y> last <Z>

T5 Models and Performance

❖ models

- Encoder-Decoder: T5 main baseline model
- Performance: Encoder-decoder > prefix LM > LM



Architecture	Params	Cost	GLUE	CNN3M	SQuAD	SGLUE	EnDe	EnFr	EnRo
★ Encoder-decoder	$2P$	M	83.28	19.24	80.88	71.36	26.98	39.82	27.65
Enc-dec, shared	P	M	82.81	18.78	80.63	70.73	26.72	39.03	27.46
Enc-dec, 6 layers	P	$M/2$	80.88	18.97	77.59	68.42	26.38	38.40	26.95
Language model	P	M	74.70	17.93	61.14	55.02	25.09	35.28	25.86
Prefix LM	P	M	81.82	18.61	78.94	68.11	26.43	37.98	27.39